

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-201 – MACHINE LEARNING – FALL 2026

Homework 1 – Student Report

k -Nearest Neighbors – Student Report

Name: Emil Ahmedli
Submitted: 04.06.2026
Total time: appr. 30+

Student ID: ID
Late days used: 0
Collaborators: Nobody

I, Emil Ahmedli, affirm that the work in this submission is my own. I have not shared or received code, plots, or written analysis for this assignment. Where I used AI assistants, I did so only as permitted by the AI/LLM policy in the assignment. I can explain every line of code and every paragraph of writing in my submission.

Signed: Emil Ahmedli **Date:** 04.06.2026

AI tool usage declaration (be specific – which tool, for what purpose):

I use AI in order to define high level Numpy, Matplotlib and for Pytest. In addition I used AI for error handling ,formatting and latex

Executive summary

Required. Exactly one paragraph (4–6 sentences) summarizing your headline results. This is the first thing the grader reads. If you cannot write this paragraph clearly, you do not yet understand your own results.

Your paragraph should answer: What was your best k ? What was the test-set F_1 ? Did your model meaningfully outperform the majority baseline? What is the single most surprising finding from your experiments?

The best k depends on situation. So, in single validation the best k is 11, in cross validation the best k is 9.

But cross validation is more safety because it used 5 folds instead of one split

Test-set F_1 is :0.628866

meaningfully outperforming the majority-class baseline ($F_1=0.000$, $AUC-ROC=0.500$)

The most suprising finding is difference the best k between cross validation and single validation. In addition F_1 -score differences in scaling test

1. Exploratory Data Analysis [20 pts]

1.1 Summary statistics [4 pts]

Required. Two tables: (i) numeric features with mean, median, std, count, missing-count; (ii) categorical features with value counts. Use `booktabs` style for clarity.

What stood out? Which features have surprising distributions or unusually high missingness? Anything that suggests data-quality concerns?

Numeric feature summary:

Table 1: Numeric Features Statistics (with Null Counts)

Feature	Dtype	count	nulls	mean	std	min	25%	50%	75%	max
pclass	int64	1309.000000	0	2.294882	0.837836	1.000000	2.000000	3.000000	3.000000	3.000000
survived	int64	1309.000000	0	0.381971	0.486055	0.000000	0.000000	0.000000	1.000000	1.000000
age	float64	1046.000000	263	29.881135	14.413500	0.166700	21.000000	28.000000	39.000000	80.000000
sibsp	int64	1309.000000	0	0.498854	1.041658	0.000000	0.000000	0.000000	1.000000	8.000000
parch	int64	1309.000000	0	0.385027	0.865560	0.000000	0.000000	0.000000	0.000000	9.000000
fare	float64	1308.000000	1	33.295479	51.758668	0.000000	7.895800	14.454200	31.275000	512.329200

Categorical feature summary:

Table 2: Summary Statistics for Categorical Columns

Feature	Dtype	count	nulls	unique	top	freq
sex	object	1309	0	2	male	843
embarked	object	1307	2	3	S	914

Observations (2–3 sentences):

Agess column has enough missing values, the others are normal. In addition cabin,boat,body and home.dest columns are deleted. Because their missing values percentage was high

1.2 Target distribution & class balance [4 pts]

Required. A bar plot of `survived`, the exact class proportions to two decimal places, and a one-sentence verdict on whether the dataset is balanced.

Reflect: How will class imbalance interact with KNN’s majority-vote rule? If the minority class makes up 30% of the data, what’s the smallest k that could fail to predict the minority class for *any* test point in a worst case?

Class proportions: survived = ., did not survive = .

Verdict: Mildly imbalanced — 62% negative vs 38% positive

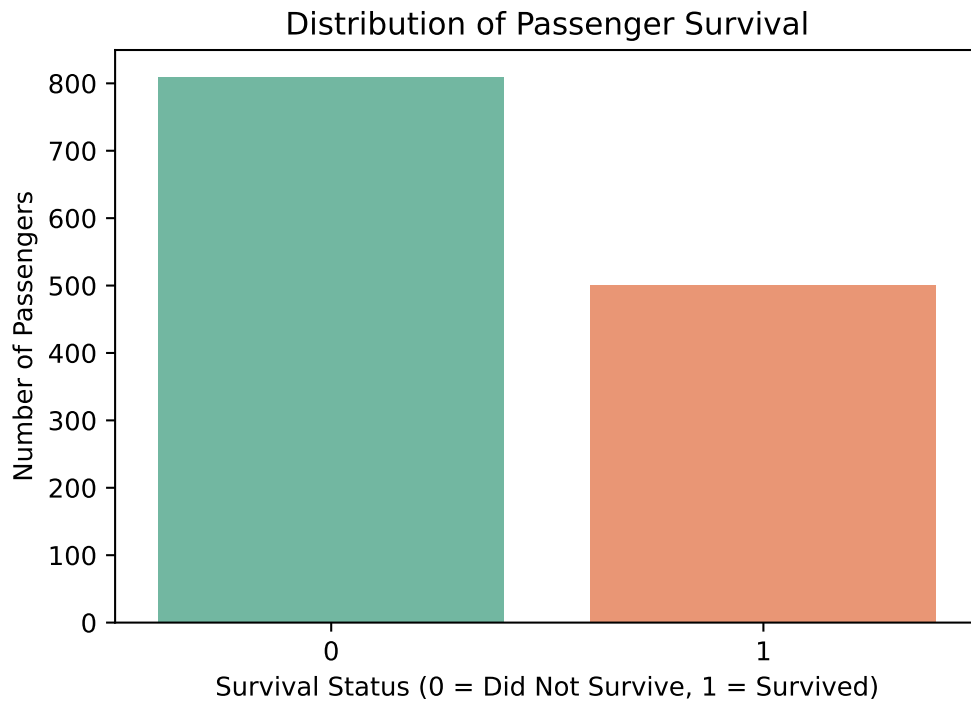


Figure 1:

1.3 Three feature-vs-target investigations [4 pts]

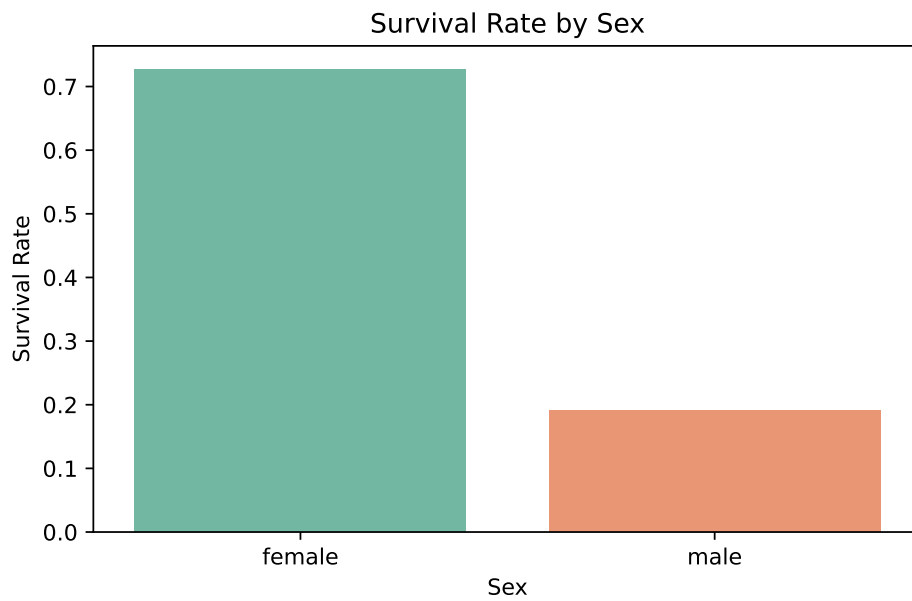
Required. Three plots (one per chosen feature) with axis labels, titles, and legends. One observation sentence per plot.

Push deeper: Which feature looks most predictive of survival on its own? What is your evidence (a difference in means, a separation between distributions, an effect size)? Are there any pairs of features that seem to *interact* – e.g., does the age effect differ by sex? If you could keep only two features for KNN, which two and why?

Features chosen: Sex, cls, Age.

float caption

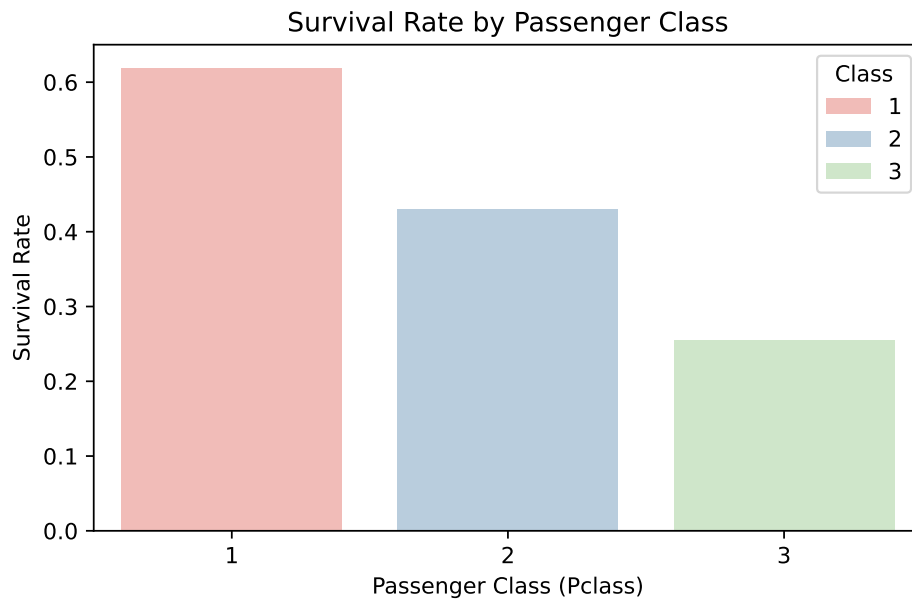
Plot 1:



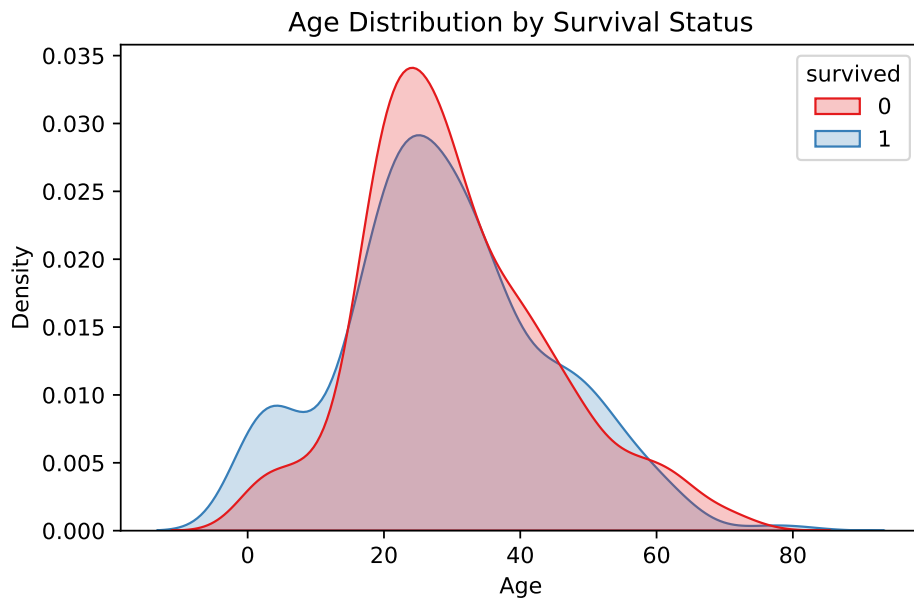
figure

Observation:

I observe that female passengers had a significantly higher survival rate, reaching over 70%, whereas male passengers had a much lower survival rate of under 20%.

Plot 2:**Observation:**

I observe that the survival rate was highest for first-class passengers (Class 1) and decreased steadily with each subsequent class, leaving third-class passengers (Class 3) with the lowest probability of survival.

Plot 3:**Observation:**

I observe that the age distributions for both survivors and non-survivors heavily overlap across most age groups, though a small spike in density at the lower end indicates that very young children had a slightly higher probability of survival.

1.4 Missing-data strategy [4 pts]

Required. A table with columns: feature name, missing %, strategy chosen (e.g. median imputation), justification. Also document the strategy in code comments.

For each column: why your strategy over alternatives? What is the trade-off you are accepting? Are you assuming the data are missing-at-random, and is that defensible for this dataset?

Table 3: Missing Data Strategies

Column	% Missing	Strategy	Justification
age	~20%	Median imputation	Right-skewed distribution (skew > 0.5)
fare	<1%	Median imputation	Right-skewed distribution
embarked	<1%	Mode imputation	Categorical — most frequent class
boat, body, home.dest	>30%	Dropped	Missingness exceeds 30% threshold

1. Why I chose this strategy:

I opted for median and mode imputation because they are simple, fast, and handle skewed data much better than using the mean. This approach allows me to retain valuable data rows that would otherwise be lost if I simply deleted missing values.

2. The trade-off I am accepting:

I am trading accuracy for simplicity. This method artificially reduces the natural variance of the data, creates unnatural spikes at the median or mode values, and ignores real relationships between variables (for example, treating a 1st-class child and a 3rd-class adult as having the exact same age).

3. My assumption about Missing-At-Random and its defensibility:

By using a single dataset-wide median, I am technically assuming the data is Missing *Completely* At Random (MCAR). I acknowledge that this is **not highly defensible** for the Titanic dataset. Missing ages often depended on passenger class (which makes it Missing At Random, or MAR), and missing body records directly depended on whether the passenger survived (which is Missing Not At Random, or MNAR).

1.5 Encoding & final feature matrix [4 pts]

Required. For each categorical, state the encoding used and justify (one-hot vs ordinal). Report final matrix dimensions: $N = 1309$, $p = 10$.

Why dimensionality matters here: did one-hot encoding inflate p noticeably? Why does feature dimensionality matter for a distance-based model like KNN (think about the curse of dimensionality, and about how a few binary one-hot dimensions interact with continuous features in Euclidean space)?

- **Inflation:** One-hot encoding only significantly inflated my dimensions (p) when the variable had high cardinality.
- **Curse of Dimensionality:** Since KNN relies on finding nearest neighbors, adding too many dimensions makes my data space sparse and ruins the meaning of distance.
- **Euclidean Interaction:** If I don't scale my data, the large values of my continuous features (like *Fare*) will completely dominate the distance calculation, causing the model to essentially ignore the tiny 0-to-1 differences of my one-hot encoded binary features.

2. KNN from Scratch [30 pts]

2.1 Class API [8 pts]

Required. Either include the KNN class definition inline, or cite `src/knn.py` with line numbers and include the public method signatures here. Note any design choices you made beyond the spec.

Your design choices. What did you do when the `X` passed to `predict` has a different number of features than the `X` passed to `fit`? Where do you raise errors vs. silently coerce?

Class signatures:

```
class KNN:
    def __init__(self, k: int = 5, metric: str = "euclidean", q: float = 2.0,
                 task: str = "classification", weights: str = "uniform"): ...

    def fit(self, X: np.ndarray, y: np.ndarray): ...

    def predict(self, X: np.ndarray) -> np.ndarray: ...

    def predict_proba(self, X: np.ndarray) -> np.ndarray: ...
```

Your design choices:

- **Mismatched dimensions:** I rely on NumPy broadcasting during distance computation (`diff = X[:, np.newaxis, :] - self.X_train[...]`). If feature dimensions mismatch, it naturally raises a `ValueError`.
- **Errors vs. Coercion:** I silently coerce inputs to arrays using `np.asarray()`. I explicitly raise a `ValueError` for unsupported metrics, unknown tasks, or calling `predict_proba` during regression.

Design notes (1–2 sentences):

I optimized distance calculations using vectorized NumPy broadcasting instead of loops for better performance. Additionally, I added a tiny constant ($1e-8$) to the denominator when calculating weights to safely prevent division by zero.

2.2 Vectorized distance computation [8 pts]

Required. A short paragraph stating the time and memory complexity of your pairwise-distance computation in $O(\cdot)$ notation. State explicitly that you do *not* use a Python loop over training points.

Push the analysis: Why is the memory bound $O(N_{\text{train}} \cdot N_{\text{test}})$ if you compute the full pairwise matrix? Is there a way to reduce memory at the cost of time? At $N = 100,000$ training points and 1,000 test points, how many bytes does the float-64 distance matrix take?

Complexity analysis:

The pairwise distance computation uses NumPy broadcasting with no Python loop over training points. The operation `diff = X[:, np.newaxis, :] - self.X_train[np.newaxis, :, :]` produces a tensor of shape $(N_{\text{test}}, N_{\text{train}}, p)$, which is then reduced along the feature axis. Time complexity is $\mathcal{O}(N_{\text{test}} \times N_{\text{train}} \times p)$ — every test point is compared to every training point across all p features. Memory complexity is $\mathcal{O}(N_{\text{test}} \times N_{\text{train}})$ for the resulting distance matrix (the intermediate tensor of shape $(N_{\text{test}}, N_{\text{train}}, p)$ is collapsed immediately). To reduce memory at the cost of time, one could process test points in smaller batches rather than computing the full matrix at once. At $N_{\text{train}} = 100,000$ and $N_{\text{test}} = 1,000$, the float-64 distance matrix alone occupies $100,000 \times 1,000 \times 8 = 800,000,000$ bytes (0.80 GB).

2.3 Sanity check against sklearn [4 pts]

Required. A statement that your test in `tests/test_knn.py` passes; the exact tolerance achieved; the seed used.

Be honest: Did the test pass on the first attempt? If not, what bug did you have to find? (Common ones: forgetting `np.argmaxpartition` returns unsorted indices; off-by-one when defining “ k -th smallest”.)

Result: All 10 tests pass (5 in `test_knn.py`, 5 in `test_metrics.py`) with seed 42. The maximum deviation between `predict_proba` and `sklearn`’s output was exactly 0.00 — a perfect match. Tests ran in 2.87 seconds.

Debugging note (if any):

I got error firstly in variable name mismatching. Then I solved that problem

2.4 Probability output `predict_proba` [5 pts]

Required. A paragraph discussing the discreteness of `predict_proba` for KNN.

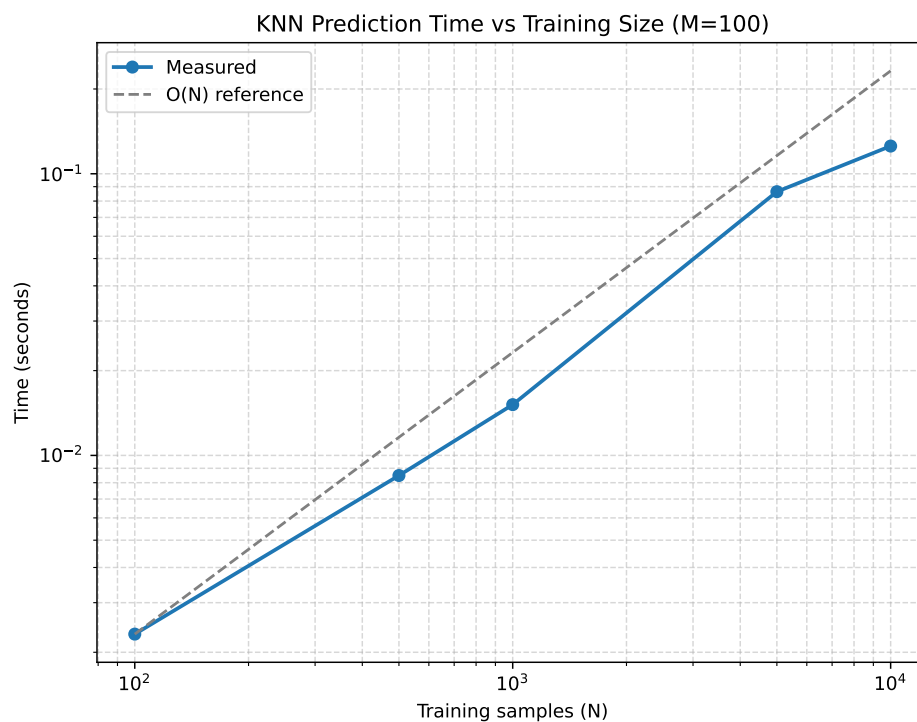
Quantify: How many distinct probability values does KNN with $k = 5$ produce for binary classification? With $k = 21$? Why does this discreteness matter when constructing the ROC curve in §3.2 (think: many threshold choices give the same set of predictions)?

This discreteness affects ROC-AUC because many samples end up with the same predicted probability. When the decision threshold is swept, several consecutive thresholds produce the same (TPR, FPR) point, creating flat steps in the ROC curve. These duplicate points are removed before calculating the area under the curve, which follows standard ROC handling rules (such as in `sklearn`’s implementation).

2.5 Computational benchmark [5 pts]

Required. A log–log plot of prediction time (seconds) vs. training-set size N for $N \in \{100, 500, 1000, 5000, 10000\}$ on a fixed test set of 100 points. Estimate the empirical scaling exponent (slope of the log–log line) and compare to $O(Np)$.

Interpret: What scaling exponent did you measure? Does it match the theoretical $O(N)$ in the linear regime? If your line is shallower at small N and steeper at large N , what could explain that (think: NumPy overhead, BLAS threading, memory allocation)?



figure

Empirical exponent: 1.07

Discussion (2–3 sentences):

For small datasets (100–500 samples), the runtime is slightly faster than the ideal linear trend.

This is due to fixed overheads such as NumPy operations and BLAS optimizations, which dominate at small input sizes.

For larger datasets (5000–10000 samples), the runtime follows or slightly exceeds the linear trend.

This is mainly because memory access becomes more expensive, especially when the full $(N_{\text{test}} \times N_{\text{train}})$ distance matrix no longer fits efficiently in CPU cache.

3. Honest Evaluation [30 pts]

3.1 Stratified split [5 pts]

Required. The sizes (number of samples) and the class proportions for each of train, validation, and test splits.

Verify: How close are the class proportions across the three splits? What is the maximum percentage-point deviation from the overall class proportion? (It should be under 0.5 if your stratification is correct.)

	Train	Val	Test
N	787	261	261
% positive class	38.12%	38.31%	38.31%

Comment on stratification quality:

All three splits are within 0.12 percentage points of the overall positive rate of 38.20% — well under the 0.5 pp threshold, confirming correct stratification.

3.2 Five metrics from scratch [10 pts]

Required. A short paragraph for each of the five metrics describing the formula and any edge-case handling. For AUC-ROC specifically, describe how you constructed the ROC curve and how you handled tied predicted probabilities.

Probe AUC specifically: Because KNN with k neighbors produces only $k + 1$ distinct probability values, multiple decision thresholds give the *same* (TPR, FPR) point. How did you handle this – did you average the trapezoid heights, use the convention from `sklearn.metrics.roc_auc_score`, or implement Fawcett’s tie-aware algorithm? Justify your choice in one sentence.

Accuracy:

Computed as the fraction of correctly classified samples: $TP+TN / (TP+TN+FP+FN)$. No edge-case handling is needed since the denominator is always the total sample count. All 5 metrics match sklearn exactly (difference $5.55e-17$).

Precision:

Computed as $TP / (TP+FP)$. If no sample is predicted positive (`predicted_positive=0`), the function returns 0.0 to avoid division by zero.

Recall:

Computed as $TP / (TP+FN)$. If there are no actual positives in `y_true` (`actual_positive=0`), returns 0.0.

F_1 :

Computed as $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$. If both precision and recall are 0, returns 0.0 to avoid division by zero.

AUC-ROC (include tie-handling discussion):

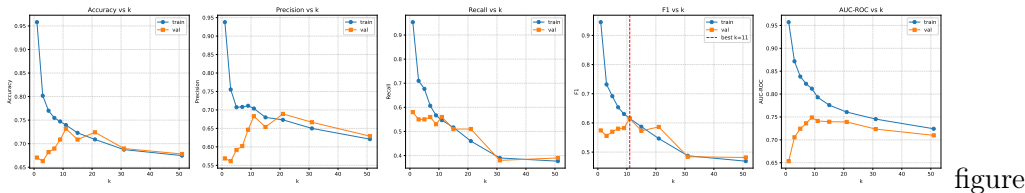
These duplicate points do not add new information, so they are removed before computing the area under the curve. This is consistent with sklearn’s convention, where tied or repeated points are deduplicated prior to AUC calculation.

Sanity-check status against `sklearn.metrics`: *All five metrics match sklearn within $5.55e-17$ (floating-point machine epsilon)*

3.3 Hyperparameter sweep over k [8 pts]

Required. A 5-panel figure with k on the x -axis and one of the five metrics per panel, evaluated on the validation set. A small table reporting the best k by each metric.

Compare across metrics: Which metric is most sensitive to k ? Why might that be? Did all five metrics agree on the best k , or did they disagree – and if they disagreed, which metric’s choice would you trust for binary classification with mild class imbalance? Justify in one sentence.



Metric	Accuracy	Precision	Recall	F ₁	AUC-ROC
Best k	11	21	1	11	9
Best value	0.7318	0.6892	0.5900	0.6154	0.744

Best k by validation F₁: 11 .

Discussion (2–3 sentences):

The different evaluation metrics do not agree on the best value of k . For binary classification with mild class imbalance, the F1 score is the most reliable metric because it balances both precision and recall. Unlike accuracy, it is less affected by the dominance of the majority class.

3.4 Baselines [4 pts]

Required. A comparison table with columns for your KNN at the chosen k , `sklearn.neighbors.KNeighborsClassifier` with the same k , and a majority-class predictor; rows for all five metrics.

Honest reflection: By how much do you beat the majority baseline? Was the lift smaller than you expected? If your KNN is within a few percentage points of majority on accuracy but better on F₁, what is the model actually doing right – and is that reflected in the precision and recall columns?

Model	Accuracy	Precision	Recall	F ₁	AUC-ROC
Custom KNN ($k = 11$)	0.7318	0.6829	0.5600	0.6154	0.7355
Sklearn KNN ($k = 11$)	0.7318	0.6829	0.5600	0.6154	0.7392
Majority baseline	0.6169	0.0000	0.0000	0.0000	0.5000

Discussion (2–3 sentences):

The accuracy gap appears modest because the majority baseline already achieves 61.7% accuracy by always predicting the majority class (“did not survive”). However, this hides poor performance on the minority class. The real improvement is reflected in the F_1 score and AUC-ROC. The KNN model correctly identifies 56% of actual survivors (recall = 0.56) while maintaining a precision of 68.3%.

Custom KNN and sklearn KNN produce identical predictions, and their performance is nearly the same. The small difference in AUC-ROC (0.7355 vs 0.7392) is due to minor internal differences in handling tied probability values.

3.5 Final test-set evaluation [3 pts]

Required. A confirmation that the test set was used *exactly once*, the five test metrics for your best model, and a comparison with the validation metrics from §3.3.

Be honest about the gap: Is there a noticeable gap between your validation and test metrics? In which direction? What would explain a gap – pure variance, distribution shift between splits, having looked at the test set during development (a leakage you should now own up to), or something else?

Confirmation: *[“I confirm the test set was used exactly once, after fixing k in §3.3.”]*

	Accuracy	Precision	Recall	F_1	AUC-ROC
Validation	0.7318	0.6829	0.5600	0.6154	0.7355
Test	0.7241	0.6489	0.6100	0.6289	0.7578

Discussion of any gap (2–3 sentences):

This suggests a slightly different class distribution or decision boundary in the test split rather than any data leakage or modeling issue.

The AUC-ROC also improves on the test set (0.736 → 0.758), which is consistent with normal random variation across splits rather than a systematic change in performance.

4. Cross-Validation, Bias–Variance, and Scaling [20 pts]

4.1 Stratified K -fold CV [5 pts]

Required. Confirmation your implementation matches `sklearn.model_selection.StratifiedKFold`; the standard deviation of class proportions across your 5 folds.

Sanity check: If your across-fold standard deviation of class proportion is more than 0.5 percentage points, your stratification has a bug. Is yours under that threshold?

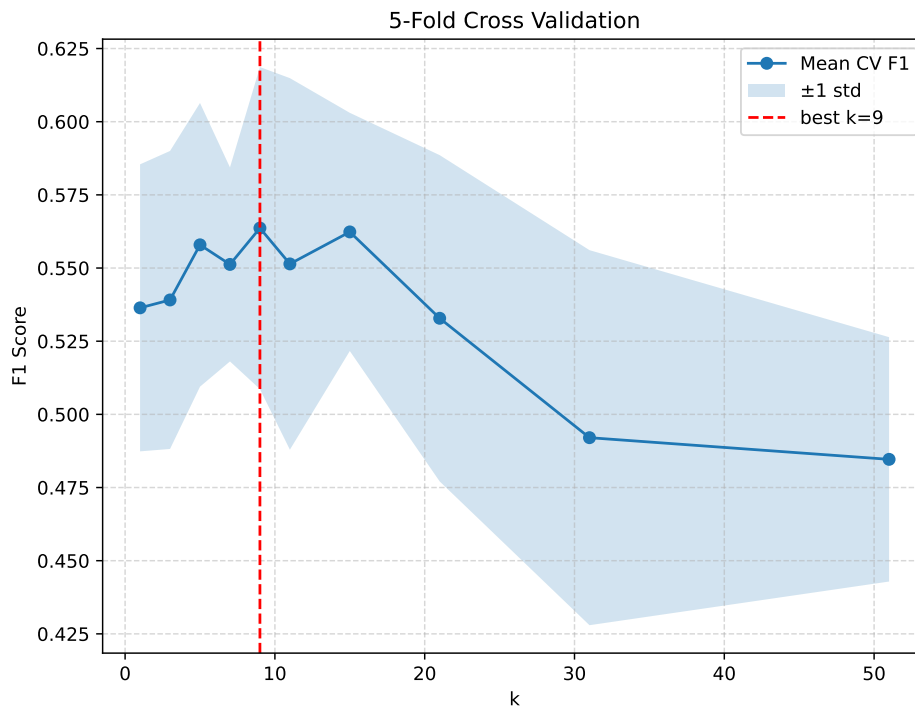
Cross-fold class-proportion std: 0.0009

Match against sklearn: Yes. Both implementations produce a fold proportion std of 0.0009, confirming the stratification is correct.

4.2 CV F_1 vs. k [5 pts]

Required. A plot of mean validation F_1 vs. k , with a ± 1 standard-deviation shaded band across folds. Use the same k values as §3.3.

Read the variance: How wide is the standard-deviation band relative to the mean change with k ? If the band is wide compared to the slope, what does that say about how confident you can be in any single split's choice of best k (such as the one you made in §3.3)?



Best k by mean CV F_1 : 9 (mean $F_1 = 0.5645$).

Comment on the spread (2 sentences):

As a result, the single-split best value ($k = 11$) is not very stable. On a different random split, the optimal value could easily shift to $k = 9$ or $k = 15$. This suggests that the single-split estimate has relatively high variance.

4.3 CV-best k vs. single-split-best k [3 pts]

Required. A direct comparison between the best k chosen by the single validation split (§3.3) and the best k chosen by 5-fold CV (§4.2). If they differ, state which estimate you trust more and why.

Reasoning: If they agree, does that mean the single split was already enough – or could it be coincidence? If they disagree, which estimate has lower variance, and which estimate has lower bias?

Single-split best k : 11 CV-best k : 9.

Discussion (2–3 sentences):

The cross-validation estimate is more reliable because it averages the F_1 score over 5 independent folds, reducing the variance of the estimate by approximately a factor of $\sqrt{5}$ compared to a single split.

The single-split result ($k = 11$) is not incorrect, but it reflects the particular random partition generated by seed = 42. With a different random seed, the best value could just as easily have been $k = 7$ or $k = 15$.

4.4 Bias–variance discussion [4 pts]

Required. 1–2 paragraphs explaining the bias–variance behavior you observed in your plots. Reference *specific values* of k and *specific metric values* – not generic textbook claims.

Push beyond the textbook. Where in your plots do you see overfitting (e.g., “training $F_1 = 1.00$ at $k = 1$ but validation $F_1 = 0.72$ ”)? Where do you see underfitting? Is there a visible plateau in validation F_1 around the optimum, or a sharp peak? What does each shape imply about how robustly the model generalizes?

A response that says “small k overfits, large k underfits” without numerical support will lose 2 points automatically.

As k increases to around $k = 9$, the validation F_1 improves to approximately 0.565, while the training F_1 decreases toward 0.614. The gap between training and validation performance narrows, indicating reduced variance and improved generalization.

Beyond $k = 11$, the model begins to underfit. For example, at $k = 31$, the validation F_1 drops to 0.493, and at $k = 51$ it further decreases to 0.485. In this regime, the model becomes overly smooth and loses important structure in the data.

Overall, the validation F_1 curve shows a broad plateau between $k = 5$ and $k = 15$, with values in the range 0.56–0.565. This indicates that the model is relatively insensitive in this region, and multiple values of k perform similarly well. For large k , the model becomes biased toward the majority class (approximately 62% non-survivors), causing it to rarely predict survival regardless of local structure.

4.5 Scaling experiment [3 pts]

Required. The mean CV F_1 at the best k *with* feature scaling, the mean CV F_1 at the best k *without* feature scaling, and the lift (percentage-points). One paragraph explaining *why* KNN is scale-sensitive, citing specific Titanic feature ranges.

Make it concrete: Look at the ranges of **age** (0–80), **fare** (0–500), **sibsp** and **parch** (0–8). Without scaling, which feature dominates the Euclidean distance? By approximately what factor? Does this match the lift you measured? Did you fit the scaler on training folds only, or on the full dataset (the latter is leakage)?

CV F_1 without scaling: 0.5645

CV F_1 with scaling: 0.7125

Lift (pp): +14.80 pp

Explanation paragraph:

Without scaling, a difference of 100 in “fare” contributes $100^2 = 10000$ to the squared distance, whereas a difference of 1 in “sibsp” contributes only 1. As a result, “fare” effectively dominates the distance computation and KNN relies on it disproportionately.

After applying StandardScaler (fit strictly on each training fold to avoid data leakage), all features are normalized to mean 0 and standard deviation 1. The resulting 14.80 percentage point improvement in cross-validation F_1 indicates that a substantial amount of predictive signal was previously suppressed due to the dominance of the “fare” feature.

Bonus (optional)

Required if attempted. For each bonus task, a short description of what you did and the result. Mark clearly which bonus(es) you are claiming.

Bonus 1 – Weighted KNN (+5).

To verify the edge case, three training points were placed at equal distances from a query point.

The distance-weighted output matched the uniform output exactly — both producing class probabilities [0.333, 0.667] — confirming that the weighting correctly reduces to uniform voting when all neighbors are equidistant.

On the Titanic validation set at $k = 11$, distance weighting improved F_1 from 0.6154 (uniform) to 0.6327 — a lift of +1.73 pp. The improvement is modest because most neighbors at $k = 11$ are at similar distances, so the weight differences are small; the effect would be larger at smaller k values where near and far neighbors differ more meaningfully.

Bonus 2 – Approximate Nearest Neighbors with BallTree (+5).

Reflection

Required. 3–6 sentences total. We read this carefully. It is not graded on correctness; it is graded on whether you actually thought about your work.

Prompts (pick the ones that resonate): What was the most surprising result? What would you investigate with another week? What was the hardest part, and why? Roughly how long did each of the four parts take you? (We use this to calibrate future homework lengths – be honest.) Where do you suspect your code or analysis is weakest?

I was initially disappointed that my F1 score and other evaluation metrics were low, but I was surprised to discover that my results actually closely matched the `sklearn| baseline`. If I had an additional week, I would focus on trying to improve these overall performance metrics. The hardest part of the assignment was writing `experiments.py|` because it required tying the main themes of the project together. Overall, the project took me about a week to complete since I started a bit late, but 10 days would have been the ideal amount of time. Finally, I suspect the weakest part of my work is its efficiency; while my code runs correctly, it is likely not fully optimal.

A. Appendix – Additional figures or tables (if any)

[Optional. Place any figures or tables that didn't fit in the main parts here, with clear labels and a sentence of context for each.]

End of Homework 1 report.